CPT102-2526 Exercise 10 Shortest Paths and Minimum Spanning Trees - Soln

In Lecture 10, we have solved the *single-source shortest paths* problem using **Dijkstra's** algorithm, and *minimum spanning trees* using **Prim's** and **Kruskal's** algorithms. You will now use them to solve other problems in this exercise.

You may write your algorithm in either pseudocode or Java. Then, justify its correctness and explain how it satisfies the required worst-case running time.

Problem 10.1 Shortest Path with a Skip

Given an edge-weighted digraph with nonnegative weights, find the shortest path from s to t where you have the option to change the weight of any one edge to 0. The required worst-case running time is **O(E log V)**.

Hint: Run Dijkstra's algorithm as the first step.

Solution:

Run Dijsktra's to compute the shortest path from s to every other vertex, and run Dijkstra's the second time to compute the shortest path from every vertex to t. For each edge e = (v, w), compute the sum of the length of the shortest path from s to v and the length of the shortest path from w to t, effectively ignoring the weight of e. The smallest such sum provides the shortest such path. Since we only run Dijkstra's two times, the running time is O(E log V).

Problem 10.2 Edge Disjoint Paths

Given a directed graph with non-negative edge weights and two vertices s and t, find two edge-disjoint paths (paths that do not share any common edges) from s to t such that the sum of the weights of the two paths is minimized, in **O(E log V)** time.

Hint: This requires running Dijkstra's algorithm twice.

Solution:

Use the first shortest path, then find the cheapest way to augment it without sharing directed edges.

```
Input: directed graph G=(V, E), nonnegative weight w(e), source s, target t
Run Dijkstra from s in G; store dist[x] and parent[x]
If t is unreachable, report that no two paths exist
Let P1 be the s-to-t path found from the parent links
```

```
For every edge (u, v), def adjustedWeight(u, v) = w(u, v) + dist[u] - dist[v]
Build a residual graph R as follows:
    If edge (u, v) is in P1, add only reverse edge (v, u) to R with weight 0
    Otherwise add original edge (u, v) to R with weight adjustedWeight(u, v)
Run Dijkstra from s to t in R using these residual weights
If t is unreachable in R, report that no two edge-disjoint paths exist
Let P2 be the path found in R
Start with the set of directed edges in P1
For each edge e in P2:
    If e is the reverse of edge f already in the set, remove f from the set
    Otherwise add e to the set
Extract two paths by starting at s twice and following unused outgoing edges
until t
Return the two extracted paths and their total original weight
```

The adjusted weights are nonnegative because shortest-path distances satisfy dist[v] <= dist[u] + w(u, v). For any complete s-to-t path, the total adjusted weight equals its original weight plus the same constant dist[s] - dist[t], so shortest s-to-t choices are preserved. Reversing the edges of P1 represents canceling any edge that would otherwise be shared with the first path. The second Dijkstra run therefore finds the cheapest augmentation of one existing s-to-t path into two edge-disjoint s-to-t paths. After opposite directed copies are canceled, the remaining edges contain exactly two directed s-to-t paths, and no edge is used by both. Since each step chooses the cheapest valid augmentation under equivalent weights, the returned pair has minimum total original weight.

There are two Dijkstra runs, each O(E log V). Reweighting edges, building the residual graph, canceling edges, and extracting the two paths are all linear in the graph size. Thus the total running time is O(E log V).

Problem 10.3 Maximum Spanning Trees

Show how to find **a maximum spanning tree** of an edge-weighted graph in **O(E log V)**.

Hint: Modify Prim's algorithm.

Solution:

Use the same growing-tree idea as Prim, but keep for each outside vertex only its best edge into the tree.

```
Input: connected undirected weighted graph G = (V, E)
Choose any start vertex r
tree <- empty set of edges
marked[x] <- false for every vertex x
bestWeight[x] <- -infinity and bestEdge[x] <- none for every vertex x
bestWeight[r] <- 0; put r in an indexed max-priority queue keyed by bestWeight
While the priority queue is not empty:
```

```
    Remove a vertex v with maximum bestWeight
    Mark v
    If bestEdge[v] is not none, add bestEdge[v] to tree
    For each edge e = (v, w) incident to v:
        If w is unmarked and weight(e) > bestWeight[w], then
            bestWeight[w] <- weight(e); bestEdge[w] <- e
            Insert w into queue, or increase its key if it is already there
Return tree
```

At every step, the marked vertices define a cut between vertices already in the tree and vertices not yet in the tree. For each unmarked vertex, the queue stores the heaviest known edge from that vertex to the marked side; therefore the next removed vertex is reached by a heaviest crossing edge. Such an edge is safe for a maximum spanning tree: if a maximum spanning tree used a lighter crossing edge for the same cut, replacing it by the heaviest crossing edge would keep the graph connected and would not decrease the total weight. The algorithm adds safe edges until all vertices are reached, so the result is a maximum spanning tree.

Each edge is examined when one of its endpoints is added to the tree, giving O(E) edge scans. Each queue insert, removal, or key increase costs O(log V), and there are O(E) such operations in the worst case. The total running time is O(E log V).

Problem 10.4 Minimum Spanning Forest

Show how to find **a minimum spanning forest** (like MST, but *not* necessarily connected) of an edge-weighted graph in **O(E log V)** or **O(E log E)**.

Hint: Modify Prim's or Kruskal's algorithm.

Solution:

Run the cycle-avoiding greedy algorithm over the whole graph; disconnected components simply remain as separate trees.

```
Input: undirected weighted graph G = (V, E), not necessarily connected
forest <- empty set of edges
Make a separate union-find component for every vertex
Put all edges into a min-priority queue ordered by weight
While the priority queue is not empty:
    Remove an edge e = (u, v) of minimum weight
    If u and v are already in the same union-find component, ignore e
    Otherwise add e to forest and union the components containing u and v
Return forest
```

The algorithm never creates a cycle, because it adds an edge only when its endpoints are in different current components. When it adds an edge, that edge is the lightest available edge crossing some cut between two components of the partially built forest, so the cut property says it is safe for a minimum spanning tree of the connected component that contains it. Since no edges exist between different connected components of the original graph, each component is handled independently. Therefore the output is the union of the minimum spanning trees of all connected components, which is a minimum spanning forest.

Building and processing the min-priority queue costs O(E log E). Union-find operations are almost constant time per edge and do not dominate. Hence the running time is O(E log E).

This is the end of CPT102-2526 Exercise 10 Soln.